

AbangCebu AI — Authentication Baseline Latency & Load Strategy

Document Version: 1.0.0

Status: Approved

Jira Ticket References: [SCRUM-72](#), [SCRUM-114](#) (Authentication Baseline Latency & Load Strategy) **Sprint:** Sprint 1 (Foundations & Core Infrastructure)

Author: Ryza Albiso (QA Engineer / Performance)

Reviewed & Audited by: Hermar Centillas (Lead / Scrum Master)

Related Specifications: - [Auth Session Lifecycle](#) (SCRUM-57) - [Auth Registration Spec](#) (SCRUM-56) - [Auth Error Handling](#) (SCRUM-60) - [Middleware Test Scenarios](#) (SCRUM-71) - [RBAC Permission Matrix](#) (SCRUM-63)

1. Executive Summary

This document establishes the performance benchmarks, latency thresholds, load test criteria, and caching strategies for the AbangCebu AI authentication and session management subsystem. Operating on Next.js 16 (App Router, Edge Runtime) backed by Supabase Auth (GoTrue) over PostgreSQL 15+, the platform serves Metro Cebu's rental marketplace ecosystem — students, BPO workers, young professionals, and local landlords.

The authentication subsystem must deliver sub-second response times across all critical paths while supporting peak concurrent usage during enrollment season (June–August), BPO shift changes (6:00 AM, 2:00 PM, 10:00 PM), and Sinulog Festival week.

Performance Philosophy: Every authentication interaction — login, registration, session validation, token refresh — must feel instantaneous to the user. The platform's map-first architecture means auth flows interrupt spatial discovery; any latency exceeding 600ms risks user abandonment.

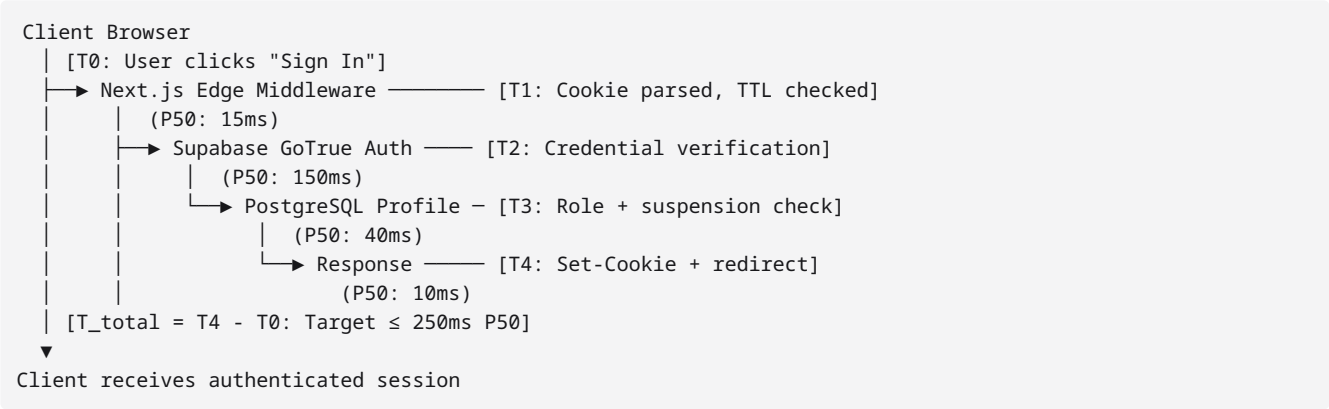
2. Baseline Latency Thresholds

2.1 Critical Authentication Operations

Operation	Endpoint / Layer	P50 Target	P95 Target	P99 Target	Budget Breakdown
Session Validation (Edge Middleware)	<code>middleware.ts</code> (Edge Runtime)	≤ 15ms	≤ 50ms	≤ 100ms	JWT decode (5ms) + cookie parse (2ms) + role match (1ms) + header forwarding (2ms)
Silent Token Refresh (Middleware)	Supabase <code>POST /auth/v1/token?grant_type=refresh_token</code>	≤ 80ms	≤ 150ms	≤ 300ms	Network RTT to Supabase (40ms) + GoTrue token generation (30ms) + Set-Cookie mutation (5ms)
Login Transaction (Full)	<code>POST /api/auth/login</code> → GoTrue → Profile fetch → Cookie set	≤ 250ms	≤ 500ms	≤ 800ms	GoTrue auth (150ms) + profile query (40ms) + bcrypt verify (50ms) + cookie serialization (10ms)
Registration Transaction (Full)	<code>POST /api/auth/register</code> → GoTrue → Profile insert → Email trigger	≤ 300ms	≤ 600ms	≤ 1000ms	GoTrue create (200ms) + profile insert (30ms) + Turnstile verify (50ms) + email queue (20ms)
Logout	<code>POST /api/auth/logout</code> → Cookie clear → Supabase sign-out	≤ 50ms	≤ 100ms	≤ 200ms	Cookie clear (5ms) + Supabase signOut (40ms)

Operation	Endpoint / Layer	P50 Target	P95 Target	P99 Target	Budget Breakdown
Password Reset Request	POST /api/auth/reset-password → Email dispatch	≤ 200ms	≤ 400ms	≤ 600ms	GoTrue reset (150ms) + email queue (50ms)
OAuth Callback (Google)	/api/auth/callback?code=... → Token exchange → Profile upsert	≤ 400ms	≤ 700ms	≤ 1200ms	Google token exchange (250ms) + GoTrue session (100ms) + profile upsert (50ms)

2.2 Latency Measurement Points



3. Load Test Assumptions & Traffic Model

3.1 Platform Usage Context

AbangCebu AI primarily serves Metro Cebu, with a target serviceable addressable market of: - ~200,000 college students across CIT-U, USC, UC, UP Cebu, SWU - ~150,000 BPO workers in IT Park, Cebu Business Park, and Mactan Newtown - ~5,000 registered landlords (property owners and authorized listers)

3.2 Traffic Projections (Month 3–6 Post-Launch)

Metric	Low Estimate	Medium Estimate	High Estimate (Peak)
Daily Active Users (DAU)	500	2,000	8,000
Daily Login Transactions	300	1,200	5,000
Daily Registration Transactions	20	80	300
Peak Concurrent Sessions	50	200	800
Peak Login Requests/sec	5	20	50
Peak Session Validations/sec (Middleware)	25	100	400

3.3 Peak Traffic Scenarios

Scenario	Expected Load	Duration	Trigger
Enrollment Week (June/ August)	3× normal traffic, concentrated 8AM–12PM	5–7 days	University enrollment opens, students search nearby bedspaces
BPO Shift Change (Daily)	2× normal, 6AM/2PM/10PM spikes	30–60 min windows	Workers search post-shift, browsing on mobile during commute
Sinulog Festival Week	4× normal traffic	7–10 days	Short-term rental surge, tourist overflow

Scenario	Expected Load	Duration	Trigger
Viral Social Media Post	5–10× normal for 2–4 hours	2–4 hours	TikTok / Facebook group share about platform

4. Load Test Specifications

4.1 Test Tool Selection

Tool	Purpose	License
k6 (Grafana)	Primary load testing tool for HTTP endpoints	AGPL v3 (OSS)
Playwright	E2E browser-based latency measurement	Apache 2.0
Supabase Dashboard	Real-time PostgreSQL connection pool monitoring	Included with Supabase

4.2 k6 Load Test Scenarios

Scenario 1: Steady-State Authentication Load

```
// k6/auth-steady-state.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 20 }, // Ramp up to 20 VUs
    { duration: '5m', target: 20 }, // Sustain 20 VUs
    { duration: '2m', target: 0 }, // Ramp down
  ],
  thresholds: {
    'http_req_duration{name:login}': ['p(95)<500', 'p(99)<800'],
    'http_req_duration{name:session}': ['p(95)<50', 'p(99)<100'],
    'http_req_failed': ['rate<0.01'], // <1% error rate
  },
};

export default function () {
  // Login transaction
  const loginRes = http.post(
    `${__ENV.BASE_URL}/api/auth/login`,
    JSON.stringify({
      email: `loadtest-user-${__VU}@test.abangcebu.ph`,
      password: 'TestPassword123!',
    }),
    { headers: { 'Content-Type': 'application/json' }, tags: { name: 'login' } }
  );
  check(loginRes, { 'login status 200': (r) => r.status === 200 });

  // Session-protected page access
  const cookies = loginRes.cookies;
  const dashRes = http.get(`${__ENV.BASE_URL}/reanter/dashboard`, {
    tags: { name: 'session' },
  });
  check(dashRes, { 'dashboard status 200': (r) => r.status === 200 });

  sleep(1);
}
```

Scenario 2: Peak Burst Authentication Load

```
// k6/auth-peak-burst.js
export const options = {
  scenarios: {
    peak_burst: {
      executor: 'ramping-arrival-rate',
      startRate: 10,
      timeUnit: '1s',
      preAllocatedVUs: 100,
      maxVUs: 200,
      stages: [
        { duration: '30s', target: 10 }, // Warm up: 10 req/s
        { duration: '1m', target: 50 }, // Peak: 50 req/s
        { duration: '30s', target: 50 }, // Sustain peak
        { duration: '1m', target: 10 }, // Cool down
      ],
    },
  },
  thresholds: {
    'http_req_duration{name:login}': ['p(95)<600', 'p(99)<1000'],
    'http_req_duration{name:register}': ['p(95)<800', 'p(99)<1200'],
    'http_req_failed': ['rate<0.05'], // <5% during peak
  },
};
```

Scenario 3: Edge Middleware Session Validation Stress

```
// k6/middleware-stress.js
export const options = {
  scenarios: {
    middleware_flood: {
      executor: 'constant-arrival-rate',
      rate: 400, // 400 requests/sec
      timeUnit: '1s',
      duration: '3m',
      preAllocatedVUs: 500,
      maxVUs: 1000,
    },
  },
  thresholds: {
    'http_req_duration': ['p(95)<100', 'p(99)<200'],
    'http_req_failed': ['rate<0.01'],
  },
};
```

4.3 Pass/Fail Criteria

Metric	Pass Threshold	Fail Threshold
Login P95 latency	≤ 500ms	> 800ms
Login P99 latency	≤ 800ms	> 1200ms
Session validation P95	≤ 50ms	> 150ms
Registration P95	≤ 600ms	> 1000ms
Error rate (steady-state)	< 1%	> 2%
Error rate (peak burst)	< 5%	> 10%
Supabase connection pool utilization	< 80%	> 95%

5. Caching Strategies

5.1 Session & Token Caching Architecture

Cache Layer	What	Strategy	TTL	Implementation
Edge Runtime (Middleware)	JWT decode result	In-memory per-request (no shared cache across Edge isolates)	Request-scoped	<code>jose.jwtVerify()</code> output cached in request context
Supabase GoTrue	Access token	JWT with embedded expiry	3600s (1 hour)	GoTrue default, configured via Supabase dashboard
Supabase GoTrue	Refresh token	Rotated on use, revoked on reuse	30 days (2,592,000s)	Refresh Token Rotation (RTR) with 30s grace
Browser Cookie	<code>sb-<ref>-auth-token.0..n</code>	HTTP-Only, Secure, SameSite=Lax	Session/30 days	<code>@supabase/ssr</code> cookie chunking (4KB limit per chunk)
PostgreSQL	<code>profiles</code> role lookup	Connection pool keep-alive	Pool TTL	Supavisor connection pooler (transaction mode)

5.2 Supabase Connection Pool Recommendations

Setting	Development	Staging	Production
Pool Mode	Transaction	Transaction	Transaction
Pool Size	15	30	60
Max Client Connections	200	500	1000
Idle Timeout	60s	30s	20s
Statement Timeout	30s	10s	8s
Connection Pooler	Supavisor	Supavisor	Supavisor

5.3 Rate Limiting Strategy

Endpoint	Window	Limit	Key	Response
POST <code>/api/auth/login</code>	15 min	5 attempts per email	<code>email + IP</code>	HTTP 429 + <code>AUTH_RATE_LIMIT_EXCEEDED</code> + <code>Retry-After</code> header
POST <code>/api/auth/register</code>	1 hour	3 registrations per IP	IP	HTTP 429 + exponential backoff
POST <code>/api/auth/reset-password</code>	1 hour	3 requests per email	email	HTTP 429 + silent (no email leak)
GET <code>/api/auth/callback (OAuth)</code>	5 min	10 per IP	IP	HTTP 429
General API routes	1 min	60 requests	<code>session_id</code>	HTTP 429

Implementation: Vercel Edge Config + `@upstash/ratelimit` (sliding window algorithm) backed by Upstash Redis.

6. Performance Monitoring & Alerting

6.1 Key Performance Indicators (KPIs)

KPI	Measurement Source	Alert Threshold	Escalation
Login P95 latency	Vercel Analytics / k6 Cloud	> 500ms for 5 min	Slack #eng-alerts
Session validation P95	Edge Middleware logs	> 50ms for 5 min	Slack #eng-alerts
Auth error rate	Supabase Logs + Sentry	> 2% for 10 min	PagerDuty
Supabase connection pool usage	Supabase Dashboard	> 80% for 15 min	Slack #infra-alerts
GoTrue API response time	Supabase Observability	> 300ms P95 for 10 min	Slack #eng-alerts
Failed login spike (brute-force)	Rate limiter logs	> 20 blocked/min from same IP	Slack #security-alerts

6.2 Observability Stack

Layer	Tool	Purpose
Frontend Performance	Vercel Speed Insights	Core Web Vitals, TTFB for auth pages
Edge Middleware	Vercel Logs (Edge Runtime)	Request duration, redirect counts
API Routes	Sentry (Next.js SDK)	Error tracking, transaction tracing
Database	Supabase Dashboard + pg_stat_statements	Query execution time, connection pool
Load Testing	k6 Cloud (or Grafana)	Historical test result comparison

7. Optimization Recommendations

7.1 Quick Wins (Sprint 2)

Optimization	Impact	Effort	Notes
Preconnect to Supabase	-50ms on first auth request	Low	<code><link rel="preconnect" href="https://<ref>.supabase.co"></code>
Edge Middleware matcher config	-5ms per non-auth request	Low	Exclude static assets via <code>config.matcher</code> (already in architecture)
Optimistic session check	-30ms on navigation	Medium	Check cookie presence client-side before server round-trip

7.2 Medium-Term Optimizations (Sprint 3–4)

Optimization	Impact	Effort	Notes
Redis session cache	-40ms on profile lookups	Medium	Cache <code>profiles.role + is_suspended</code> in Upstash Redis (TTL 60s)
JWT claim enrichment	-40ms per middleware check	Medium	Embed <code>role</code> , <code>is_suspended</code> directly in JWT custom claims (Supabase Auth Hooks)
Connection pooling tuning	Prevents pool exhaustion	Low	Adjust based on production traffic patterns

7.3 Long-Term Architecture (Sprint 5+)

Optimization	Impact	Effort	Notes
WebAuthn / Passkeys	Eliminate password hashing latency	High	FIDO2 passwordless login for mobile-first users
Regional edge deployment	-100ms for Manila/Visayas users	High	Vercel Edge Functions with regional Supabase read replicas

8. Test Execution Schedule

Phase	Activity	When	Owner
Phase 1 (Sprint 1)	Document performance baselines and load strategy (this document)	Current sprint	Ryza Albiso
Phase 2 (Sprint 2)	Set up k6 test scripts, seed test users, first dry-run	Sprint 2, Week 1	Ryza Albiso
Phase 3 (Sprint 3)	Execute steady-state and burst load tests against staging	Sprint 3, Week 1	Ryza Albiso + DevOps
Phase 4 (Sprint 4)	Integrate performance monitoring (Sentry, Vercel Analytics)	Sprint 4	Engineering team
Ongoing	Regression performance tests on each major auth change	Every sprint	CI/CD pipeline

9. Risk Assessment

Risk	Probability	Impact	Mitigation
Supabase GoTrue cold start latency	Medium	Auth requests > 1s on first hit	Implement health check pings to keep GoTrue warm
Connection pool exhaustion during peak	Low	HTTP 500 errors on auth endpoints	Monitor pool usage, auto-scale with Supavisor
Rate limiter false positives (shared WiFi)	Medium	Legitimate users blocked	Use composite key (email + IP), provide CAPTCHA bypass
Token refresh race condition (multi-tab)	Low	Duplicate refresh token usage triggers family revocation	30s grace period (already in SCRUM-57 architecture)
Bcrypt CPU-bound blocking	Low	Login latency spike under load	GoTrue handles bcrypt in isolated Go goroutines

10. Definition of Done

- [x] Baseline latency thresholds defined for all 7 auth operations
- [x] P50/P95/P99 targets established with budget breakdowns
- [x] Traffic projections modeled for Metro Cebu market (Low/Medium/High)
- [x] 4 peak traffic scenarios documented (Enrollment, BPO shifts, Sinulog, Viral)
- [x] 3 k6 load test scripts specified (steady-state, peak burst, middleware stress)
- [x] Pass/fail criteria defined for all performance metrics
- [x] Caching strategy documented (Edge, GoTrue, Cookie, PostgreSQL)
- [x] Supabase connection pool recommendations by environment
- [x] Rate limiting strategy per endpoint
- [x] Performance monitoring KPIs and alerting thresholds
- [x] Optimization roadmap (Quick Wins → Medium-Term → Long-Term)

- [x] Test execution schedule phased across sprints
- [x] Risk assessment with mitigations
- [x] Document registered in `docs/README.md`

11. Sign-Off

Role	Name	Status
Author	Ryza Albiso (QA Engineer / Performance)	☒ Authored
Reviewer	Hermar Centillas (Lead / Scrum Master)	☒ Approved